

What Is Multithreading in Java?

- Multithreading in Java is a feature that allows a program to execute multiple tasks at the same time.
- Each task runs inside its own thread, and a thread is the smallest unit of execution in a program.
- By enabling multithreading, a program can perform several operations concurrently, such as reading files, writing logs, processing user requests, or doing background calculations.
- Java supports threads through the Thread class and the Runnable interface.
- Multithreading helps applications become faster, more responsive, and capable of handling heavy workloads without freezing.
- In simple terms, multithreading means doing many things at once instead of doing everything one by one.

How Can You Create a Thread in Java?

- In Java, you can create a thread in two main ways: by extending the Thread class or by implementing the Runnable interface.
- Both approaches achieve the same result, but implementing Runnable is preferred because it allows your class to extend something else and provides better structure for multithreading.
- When you create a thread, the key method you override is the run method, which contains the code that the thread will execute.
- To actually start the thread, you call the start method, which gives control to the thread scheduler and runs the run method on a separate path of execution.

Describe the Different States of a Thread and When the State Transitions Occur

- Java threads go through a fixed set of states during their lifetime, and these states are defined inside the `Thread.State` enum.
- You can check any thread's current state by calling the `getState()` method.
- The main thread states are: **NEW**, **RUNNABLE**, **BLOCKED**, **WAITING**, **TIMED_WAITING**, and **TERMINATED**.
- Each state represents a different phase in the thread's execution lifecycle.

NEW State

- A thread is in the **NEW** state when the `Thread` object has been created, but the `start()` method has not been called yet.
- In the **NEW** state, the thread has not begun execution.

RUNNABLE State

- A thread enters the **RUNNABLE** state immediately after calling `start()`.
- **RUNNABLE** means the thread is eligible to run; it might be running or waiting for the scheduler to assign CPU time.

BLOCKED State

- A thread enters the **BLOCKED** state when it tries to enter a synchronized block or synchronized method that is already locked by another thread.
- The **BLOCKED** state means the thread is waiting for the monitor lock to become available.

WAITING State

- A thread enters the **WAITING** state when it is waiting indefinitely for another thread to perform a specific action.
- A thread enters WAITING when calling `Object.wait()` without a timeout, `Thread.join()` without a timeout, or `LockSupport.park()`.

TIMED_WAITING State

- A thread enters the **TIMED_WAITING** state when it waits for a specific amount of time.
- This happens when a thread calls methods like `Thread.sleep(milliseconds)` or calls timed versions of `wait()` or `join()`.

TERMINATED State

- A thread enters the **TERMINATED** state when its `run()` method has completed normally or when it ends because of an exception.
- TERMINATED means the thread has finished execution permanently and cannot be restarted.
-

What Is Synchronization in Java?

- Synchronization in Java is a mechanism that ensures only one thread at a time can execute a critical section of code.
- When multiple threads update the same shared variable at the same time, the program may produce unpredictable results. This situation is called a race condition.
- Synchronization prevents race conditions by allowing only one thread to access the shared data at a time.
- Java provides synchronization using the `synchronized` keyword, which can be applied to methods or code blocks.
- When a thread enters a synchronized method or block, it acquires the lock of that object.
- As long as one thread holds the lock, no other thread can enter any synchronized section that uses the same lock.

What Is the Callable Interface?

- The Callable interface in Java is similar to the Runnable interface, but it provides two important advantages.
- A Callable task returns a value after completing its execution, while a Runnable does not return anything.
- A Callable task can also throw checked exceptions, which Runnable cannot do.
- Because of these features, Callable is used when a task must produce a result, such as calculating a sum, reading a file, calling an API, or performing a long computation.

Callable and Future Working Together

- When you submit a Callable task to an ExecutorService, it immediately returns a Future object.
- The Future object represents the result that the Callable will produce in the future.
- You can use the Future to check whether the task has finished, wait for it to finish, and retrieve the final result using the get method.
- This makes Callable useful for tasks that run in the background while your program continues.

What Is ExecutorService?

- ExecutorService in Java is a framework that simplifies how we manage threads.
- Instead of creating threads manually using the Thread class, ExecutorService allows us to submit tasks and it handles thread creation, scheduling, execution, and reuse automatically.
- ExecutorService internally uses a thread pool, which is a set of worker threads created in advance and reused to execute multiple tasks.
- Using a thread pool avoids creating a new thread for every task and improves performance in applications that need concurrency.
- ExecutorService can execute Runnable tasks that do not return a value.

- It can also execute Callable tasks that return a value through a Future object.
- ExecutorService provides methods to shut down all threads safely.
- Safe shutdown ensures the application exits cleanly.
- It prevents leaving any threads running in the background.
-

What is the difference between the Runnable and Callable interfaces in Java?

- Runnable and Callable are two interfaces in Java used to represent tasks that can be executed by threads or by ExecutorService, but they are not the same.
- The first and most important difference is that a Runnable task does not return a value, while a Callable task returns a value.
- Runnable's run method always returns void.
- Callable's call method returns a value of any type because it uses generics.
- A Runnable cannot throw checked exceptions, but a Callable can throw checked exceptions.
- When we submit a Runnable to an ExecutorService, we do not get a result.
- When we submit a Callable, ExecutorService returns a Future object that represents the result of that task and allows us to retrieve it later.